

第六章RNN实验报告

2251079 隋建政

一、模型解释

1. RNN

循环神经网络（RNN），是一种用于处理序列数据的神经网络模型。与传统的前馈神经网络所不同的是，处理一个序列时当前的输出与前面的输出也有关，也就是说隐藏层之间的节点也存在链接，其输入不止包括输入层的输出还包括上一时刻当前隐藏层的输出。这种网络结构使得RNN能够对序列前面的信息进行记忆并因公到当前输出的计算中。对于隐状态的处理可以写为：

$$h_t = f(x_t, h_{t-1}) \quad (1)$$

其中 x_t 为当前输入， h_t 为 t 时刻的因状态。

2. LSTM

长短期记忆神经网络（LSTM），是一种时间递归神经网络，其适用于处理和预测时间序列中间隔和延迟相对较长的时间。LSTM最初是为了解决循环神经网络中出现的梯度消失问题而提出的，LSTM通过其精心设计的输入门、遗忘门与输出门结构，可以有效消除或增强信息到某一状态的能力，从而“记住”长期信息并加以利用。假设我们使用sigmoid作为激活函数，则三种门的计算方式可以写为：

$$\begin{aligned} I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \\ F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \\ O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \end{aligned} \quad (2)$$

采用tanh函数作为激活函数，候选记忆元的计算如下：

$$\widetilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c) \quad (3)$$

然后就可以计算当前时间步的记忆元与输出门结果：

$$\begin{aligned} C_t &= F_t \odot C_{t-1} + I_t \odot \widetilde{C}_t \\ H_t &= O_t \odot \tanh(C_t) \end{aligned} \quad (4)$$

3. GRU

门控循环单元（GRU）比LSTM稍微简化，但通常能提供同等的效果，而且计算速度明显更快。模型可以通过门控单元学习何时更新隐状态以及何时重置隐状态。GRU主要包含重置门与更新门两种门控单元：

$$\begin{aligned} R_t &= \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r) \\ Z_t &= \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z) \end{aligned} \quad (5)$$

通过重置门可以计算候选隐状态：

$$\widetilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h) \quad (6)$$

候选隐状态与更新门可以计算当前隐状态：

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \widetilde{H}_t \quad (7)$$

二、诗歌生成过程

1. 数据处理

首先对诗歌数据进行处理，读取诗歌文本文件并按内容进行分割，将诗歌文件读入内容转化为字符列表，添加开头标记与结尾标记。统计所有字符出现的频率并构建词汇表，创建字符与ID的映射，并将诗歌映射成为id序列。

2. 模型构建

RNN模型定义嵌入层（将字符ID转化为向量）、循环层（处理序列数据）与全连接层（输出字符的概率分布结果），同时实现前向传播与生成下一个token的类内方法。

3. 模型训练

使用交叉熵作为损失函数，采用标准的梯度下降优化过程用于模型训练。使用了Adam优化算法，学习率设置为0.0005，进行10个epoch的训练。

三、生成结果

```
import random
def gen_sentence(start):
    cur_token = tf.constant([word2id[start]], dtype=tf.int32)
    state = [tf.random.normal(shape=(1, 128), stddev=0.5), tf.random.normal(shape=(1, 128), stddev=0.5)]
    collect = [start]

    for _ in range(50):
        cur_token, state = model.get_next_token(cur_token, state)
        next_id = cur_token.numpy()[0]
        if id2word[next_id] == end_token:
            break
        collect.append(id2word[next_id])
        cur_token = tf.constant([next_id], dtype=tf.int32)
    return ''.join(collect)

start_words=['日','红','山','夜','湖','海','月']
for word in start_words:
    poem = gen_sentence(word)
    print(poem)
```

[75] ✓ 0.0s Python

... 日暮风风落月中人。
红氍乍声啼不见，一片花声不可知。
山上山，日日风，日日风，月上花中。
夜色，不知何处人。
湖畔夜风风，一片花声不可知。
海阳州客，不得无人不可知。
月，江山上月中。

生成的结果为：

日暮风风落月中人。

红氍乍声啼不见，一片花声不可知。

山上山，日日风，日日风，月上花中。

夜色，不知何处人。

湖畔夜风风，一片花声不可知。

海阳州客，不得无人不可知。

月，江山上月中。